

MAF Workflows

Executors, edges, events, and state — the graph primitives everything else builds on

Source: <https://learn.microsoft.com/en-us/agent-framework/concepts/workflows/>

Executors: the processing units

- Executors receive typed messages, do work, and produce output messages or events
- Two kinds:
 - Custom logic components
 - AI agents (Module 1's "workflow with agent executors")
- Each handler's type annotations declare what it can send and yield — the workflow validates the graph against these at build time
 - `WorkflowContext.send_message(...)` forwards to connected executors
 - `ctx.yield_output(...)` produces a workflow output

A minimal executor

```
from agent_framework import Executor, WorkflowContext, handler

class UpperCase(Executor):
    @handler
    async def to_upper_case(self, text: str, ctx: WorkflowContext[str]) -> None:
        await ctx.send_message(text.upper())
```

Edges: how messages flow

Type	Description	Use case
Direct	Simple one-to-one connection	Linear pipelines
Conditional	Routes based on a condition function	Binary routing (if/else)
Switch-Case	Routes to different executors by condition	Multi-branch routing
Multi-Selection (Fan-out)	One executor sends to multiple targets	Parallel processing
Fan-in	Multiple executors send to one target	Aggregation

Conditional edges in code

```
builder = WorkflowBuilder(start_executor=planner)
builder.add_edge(planner, retriever)
builder.add_edge(retriever, critic)

# Loop back to the planner when the critic isn't satisfied
builder.add_edge(critic, planner, condition=lambda result: not result.approved)
# ...and a separate edge to wherever the workflow's approved path continues
```

This is the mechanic behind any revision loop a prebuilt orchestration pattern can't express.

Visualize the graph you just built

```
from agent_framework import WorkflowViz

viz = WorkflowViz(workflow)

print(viz.to_mermaid())    # paste into any Mermaid renderer
print(viz.to_digraph())    # Graphviz DOT format
viz.save_svg("workflow.svg")
```

Conditional edges render as dashed arrows labeled "conditional" — the lab's Critic → Planner loop-back is visually obvious, not just implied by the code.

DEMO

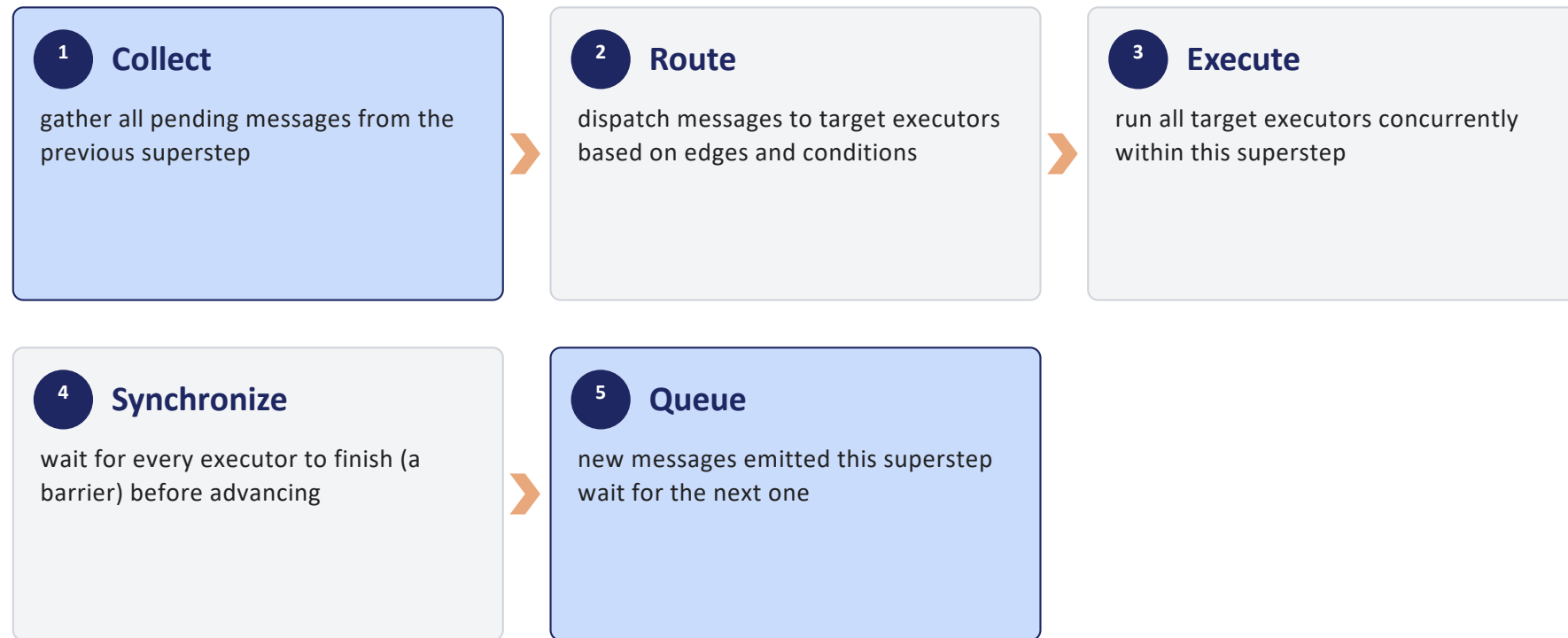
~5 min

Visualize the graph you're about to build yourself

Render the same three-executor, conditional-loop graph attendees are about to build by hand this afternoon — Planner → Retriever → Critic, with a conditional edge back to the Planner when the Critic doesn't approve. Paste the printed Mermaid into mermaid.live live, then optionally run it for real on a question that usually needs a revision pass, so the loop-back edge fires in an actual trace, not just on the diagram.

Runbook: [demos/day4/module-3-demo-1-visualize-graph.md](#)

The superstep execution model



Why supersteps matter

- **Deterministic execution** — the same input always executes in the same order
- **Reliable checkpointing** — state can be saved at superstep boundaries with no ambiguity
- **Simpler reasoning** — no race conditions between supersteps; every executor sees a consistent view

Caveat: the barrier means one slow executor blocks the whole superstep — consolidate sequential logic into one executor if independent branches need independent timing

Events give you observability

Event type	Meaning
output / intermediate	Terminal vs. observational workflow output
executor_invoked / executor_completed / executor_failed	Per-executor lifecycle
superstep_started / superstep_completed	Superstep boundaries
request_info	An executor is requesting external input (human-in-the-loop)

Debugging with edge delivery status

edge_group.delivery_status	Meaning
delivered	Message reached the target executor
dropped condition false	A conditional edge's condition evaluated false
dropped type mismatch	Target executor can't handle this message type
buffered	Waiting on more messages to arrive for a fan-in

Every edge decision in the graph — including the lab's own Critic → Planner condition — shows up here, on a span you can inspect with the same OpenTelemetry tooling from Day 3's observability content.

State: access and share common data

- `ctx.set_state(key, value)` / `ctx.get_state(key)` — state that is available to downstream executors
- Use a consistent key across executors to exchange state that isn't a direct message
- Writes are visible to the writer immediately; other executors see them starting the next superstep
- Isolation risk: reusing one workflow/executor instance across unrelated runs leaks state between them
 - Build fresh instances per run

Human-in-the-loop: request and response

- `ctx.request_info(request_data=..., response_type=...)` pauses the workflow and emits a `request_info` event
- Handle the response with a `@response_handler` method — the framework matches responses to handlers by type annotation
- This is the exact mechanism a human reviewer, an approval system, or any external system plugs into
- Same request/response channel — different payload — is how tool approval works inside a workflow

Tool approval reuses the same channel

```
@tool(approval_mode="always_require")
def execute_database_query(query: str) -> str:
    return f"Query executed successfully: {query}"

# The workflow pauses and emits a request_info event carrying a
# function_approval_request payload when the agent tries to call it
async for event in stream:
    if event.type == "request_info" and event.data.type == "function_approval_request":
        responses[event.request_id] = event.data.to_function_approval_response(approved=True)
```

Checkpoints: save and resume

- A checkpoint is created:
 - At the start of the first superstep
 - At the end of every superstep, after all its executors finish
- Captures: executor state, pending messages for the next superstep, pending requests and responses, and shared state
- Pending human-in-the-loop requests are re-emitted as `request_info` events when you restore — you never lose an outstanding approval

Use cases: long-running workflows, pause/resume across process restarts, audit/compliance snapshots, migrating a run across environments

A note on the functional API

WorkflowBuilder (today's focus)

- Explicit edges and conditions
- Fixed graphs, fan-out/fan-in, type-validated routing
- Available in Python, C#, and Go

@workflow — Python only, EXPERIMENTAL

- Native Python control flow: if, while, `asyncio.gather`
- Could simplify a custom revision loop to a plain while — worth revisiting once it's no longer experimental
- Not used for today's lab while it remains experimental

Declarative workflows: describe what, not how

- Declarative workflows define workflow logic in YAML instead of programmatic code — easier to read, modify, and share across teams
- You describe what the workflow should do, not how to implement it — the framework converts the YAML into an executable workflow graph
- Key benefits:
 - Readable — YAML syntax is easy to understand, even for non-developers
 - Portable — definitions can be shared, versioned, and modified without code changes
 - Rapid iteration — modify workflow behavior by editing configuration files
 - Consistent structure — predefined action types keep workflows following best practices

When to use declarative vs. programmatic workflows

Scenario	Recommended Approach
Standard orchestration patterns	Declarative
Workflows that change frequently	Declarative
Non-developers need to modify workflows	Declarative
Complex custom logic	Programmatic
Maximum flexibility and control	Programmatic
Integration with existing Python code	Programmatic

Python YAML structure

```
name: my-workflow
description: A brief description of what this workflow does

inputs:
  parameterName:
    type: string
    description: Description of the parameter

actions:
  - kind: ActionType
    id: unique_action_id
    displayName: Human readable name
    # Action-specific properties
```

Only name and actions are required — description and inputs are optional, and actions is the ordered list of steps the workflow actually runs.

Takeaways

- ✓ You build executors from typed message handlers, and connect them with direct, conditional, switch-case, or fan-in/fan-out edges.
- ✓ Conditional edges are what a revision loop needs — no prebuilt orchestration pattern supports routing back to an earlier agent.
- ✓ The superstep (Pregel/BSP) model is why parallel execution is real, checkpointing is reliable, and execution is deterministic.
- ✓ Human-in-the-loop and tool approval share one request/response mechanism; checkpoints preserve pending requests across a restore.
- ✓ You can render any graph you build with WorkflowViz, and diagnose a silent edge with `edge_group.delivery_status` — both work on the lab's own conditional edge.
- ✓ WorkflowBuilder is today's lab tool; the functional `@workflow` API remains experimental and worth a second look later, while declarative YAML workflows — covered on the previous three slides — are a real production alternative once a graph settles into a standard, frequently-edited pattern.